
CacheMAS: Cache Communication for Single-Pass, Jointly-Optimized Multi-Agent Systems

Chao Ouyang Yuyang Bai Jun Zhang* Tianlu Gao
Yuxin Dai Peidong Xu David Wenzhong Gao

School of Electrical Engineering and Automation, Wuhan University

Abstract

Multi-agent LLM systems (MAS) improve reasoning by decomposing across specialized roles, but their text communication is expensive. Each agent decodes thousands of intermediate tokens for the next, and successive prefills grow cumulatively. Recent work shows that latent-space signals can substitute for explicit text between agents, cutting cumulative prefill cost. When the latent reasoning also replaces an agent’s text reasoning, it additionally cuts intermediate tokens. Here, we find that the prefill cache alone suffices as the latent-space signal between agents, and the per-agent forward chains can be compressed into a single pass. With *exactly one prefill and one decode per problem*, we sharply cut per-rollout tokens and prefill wall time. We call this architecture **CacheMAS**, in which prefill agents pass information through cache communication, and a final agent decodes the answer. As a bonus, **CacheMAS** is one autoregressive rollout, so unlike prior MAS, standard sequence-level RL can jointly optimize all agents at once without per-agent reward design. On math, multi-hop QA, and code, CacheMAS uses $\sim 3\times$ fewer tokens, runs $\sim 2\times$ faster, and after joint optimization beats prior MAS by +1 to +8 points on every benchmark. Cache communication is not just a token-efficient substitute for text, but a practical interface for joint MAS optimization.

1 Introduction

Multi-agent systems (MAS) built on large language models can improve complex reasoning by decomposing problems across specialized roles [Wu et al., 2024, Hong et al., 2024, Du et al., 2023, Shinn et al., 2023], usually with one agent analyzing the problem, another drafting a solution, and a third verifying it. To pass information between these roles, agents communicate in text. Each agent decodes a response, and the next agent prefills it together with its own prompt. While this text interface is simple to implement, it is expensive in both token volume and wall time. A typical three-agent MAS rollout on AIME consumes roughly 39,000 tokens per problem, most of which serve only as input to the next agent. Successive prefills also grow longer as text accumulates, making later agent prefills increasingly slow as they process the accumulated context from earlier outputs.

Recent latent MAS frameworks let agents communicate through latent-space signals instead of explicit text. ThoughtCommunication [Zheng et al., 2025] replaces the inter-agent text channel, training an autoencoder to extract a latent thought from the hidden state of each agent’s last output token and an adapter to inject it into the next agent’s input. This saves cumulative prefill cost while each agent’s own generation still runs as text. Another approach [Zou et al., 2025] extends this by replacing each agent’s text decoding with a latent reasoning loop and passing the resulting KV cache to the next agent, cutting intermediate tokens. By reaching text-MAS accuracy training-free, it shows that the KV cache itself suffices to carry latent thoughts, further advancing latent space MAS. But

*Corresponding author. jun.zhang.ee@whu.edu.cn

this cache now mixes two kinds of content (prefill-pass KV laid down once plus per-step latent KV from the loop). Given that latent reasoning typically benefits from training to operate in latent space, a natural question is which content in cache is actually carrying the inter-agent signal.

Our controlled study on GSM8K (Section 2) answers this question. The prefill-pass KV cache alone, not the per-step latent transitions, carries the dominant inter-agent signal. Keeping only the prefill KV reaches accuracy close to the best multi-step latent setting, while keeping only the latent-step KV collapses accuracy. The latent transition states themselves drift outside the natural-language regime the base model was trained on, which helps explain why adding more of them does not help. The structural implication is that the latent interface can be reduced to its prefill-KV channel.

Acting on this finding, we drop the latent reasoning steps. Unlike prior MAS that run multiple sequential per-agent prefill-and-decode chains, this lets the pipeline collapse into a single pass with *exactly one prefill and one decode per problem*. We call this single-pass architecture **CacheMAS** (Figure 1). In CacheMAS, the three role-specialized agents run a single concatenated prefill, with each agent’s tokens routed to its own weights. The prefill produces KV cache as the only inter-agent communication interface, followed by a single decode loop where only the final agent produces the answer. This sharply cuts per-rollout tokens and prefill wall time. As a bonus, the pipeline is one autoregressive rollout with end-to-end gradient flow, unlike prior MAS that detach the gradient between agents in generation loop, making it jointly optimizable with sequence-level RL [Shao et al., 2024]. All agents update in one joint forward-backward pass per problem, sidestepping the per-agent reward shaping [Feng et al., 2026] and separate per-agent optimization [Liu et al., 2025] that prior MAS RL typically required, and cutting RL training time.

We evaluate CacheMAS on math, multi-hop QA, and code using Qwen3-8B and Hunyuan-1.8B. Even before training, CacheMAS uses about $3\times$ fewer tokens and runs about $2\times$ faster than text-based MAS, matching or exceeding it on multi-hop QA and code, competitive on math, and on par with LatentMAS without its latent loop. Joint optimization then lifts CacheMAS above text-based MAS on both models, with Qwen3-8B gains of +3 on math, +6 on multi-hop QA, and +3 on code, and larger gains on Hunyuan-1.8B. Our contributions can be summarized as follows:

- We disentangle the moving parts of latent multi-agent communication and show, in a controlled GSM8K study, that the prefill-pass KV cache, not per-step latent transitions, is the dominant carrier of the useful inter-agent signal (Section 2).
- We propose **CacheMAS**, a single-pass multi-agent architecture based on *cache communication* that compresses prior MAS’s multiple sequential per-agent prefill-and-decode chains into exactly one prefill and one decode per problem, sharply cutting per-rollout tokens and prefill wall time (Section 3.1).
- As a direct consequence of the single-pass design, the whole multi-agent pipeline becomes jointly optimizable with standard sequence-level RL at single-pass per-example cost, without per-agent reward design (Section 3.2).
- We validate CacheMAS empirically on math, multi-hop QA, and code using Qwen3-8B and Hunyuan-1.8B, where training-free CacheMAS is already competitive with prior MAS and joint optimization lifts it above them on every benchmark (Section 4).

2 Rethinking Latent Communication

To find out whether the inter-agent signal of latent MAS comes from the prefill-pass KV cache or the per-step latent KV cache, we run a controlled study with Qwen3-8B [Yang et al., 2025] on

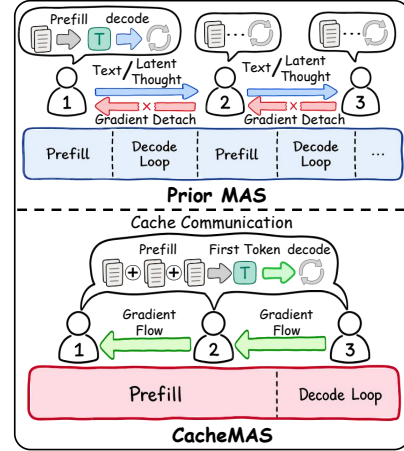


Figure 1: Prior MAS chains multiple prefill-and-decode passes. CacheMAS collapses them into a single prefill-then-decode pass with cache communication and end-to-end gradient flow.

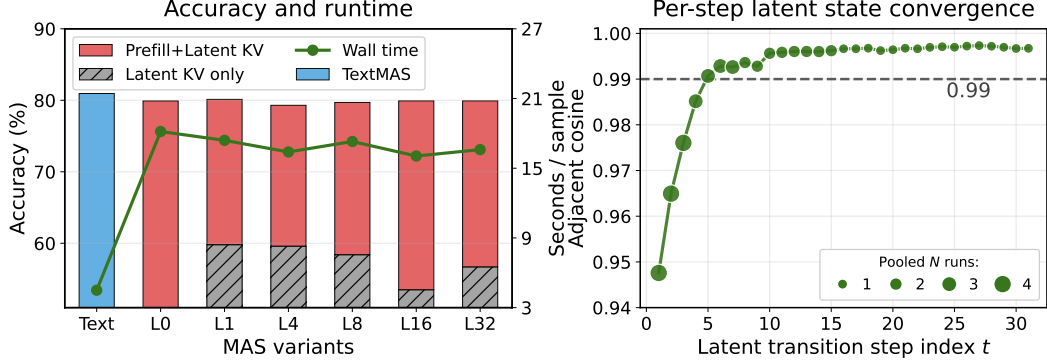


Figure 2: **Latent step utility.** **Left:** each bar at L1–L32 latent steps stacks *Latent-only* accuracy (hatched grey) below the prefill-cache contribution (red), summing to full latent-MAS accuracy. **Right:** adjacent cosine pooled across N runs at each step, with marker size reflecting N .

GSM8K-488 [Cobbe et al., 2021] (488-sample test split, exact-match accuracy), using a three-agent stack (Analyzer, Solver, Validator) where each agent passes its prefill and latent reasoning KV cache to the next instead of sampled text, following recent latent MAS frameworks [Zou et al., 2025].

2.1 The prefill-pass KV cache is the dominant carrier

We first compare TextMAS against the latent MAS baseline across $N \in \{1, 4, 8, 16, 32\}$ latent steps. The latent baseline stays within ~ 1 point of TextMAS (81%; Figure 2, left, solid bars), reproducing the prior finding that explicit text is not necessary. However, accuracy does not improve monotonically with more latent steps, hinting that the latent loop might contribute little. To check this, we measure the adjacent cosine similarity between successive latent states. The latent loop indeed saturates early, with cosine reaching ≥ 0.99 by $t \approx 5$ and plateauing thereafter (Figure 2, right). Each additional step therefore contributes almost no new information. We then run two ablations to test which component is actually carrying the signal (Figure 2, left, hatched bars). *Prefill-KV-only* reaches 79.9%, within 1 point of every full-baseline setting, while *Latent-only* collapses to 53.5%–59.8%. The asymmetry is sharp. The prefill-pass cache alone retains essentially all of the baseline’s accuracy, while the latent-step content alone retains only about two-thirds. So with untrained latent reasoning, the useful inter-agent signal lives in the prefill-pass cache, not in the latent updates.

2.2 Latent transition states drift from natural decoded-token states

Why does the latent-step content contribute so little? A natural answer is distribution shift. Without explicit training to operate in latent space, the latent states drift away from the distribution of natural decoded-token states the LLM was trained on, leaving other agents unable to interpret them as reasoning content. Two analyses confirm this. First, in a shared 2D projection of final-layer hidden states (Figure 3), latent transition states and natural decoded tokens occupy disjoint regions. Second, projecting latent states through the model’s lm-head returns almost exclusively whitespace and chat-template tokens. In our experiments, almost every top-1 token is `\n\n`, with ranks 2–5 filled by chat-template tokens (`<|im_end|>`, `</think>`, `<|im_start|>`) and no content token in any top-5. The lm-head thus sees latent states as a paragraph break rather than reasoning tokens, and scaling N does not help because each step nudges the state along a direction the LLM cannot interpret as reasoning.

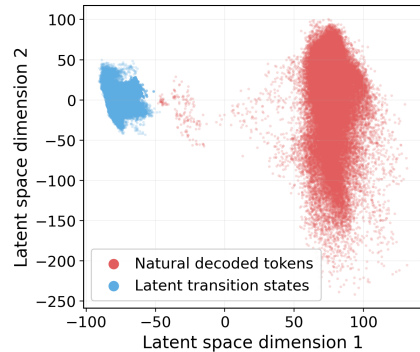


Figure 3: Distribution shift between latent transition states and natural decoded tokens on GSM8K-488 (Qwen3-8B).

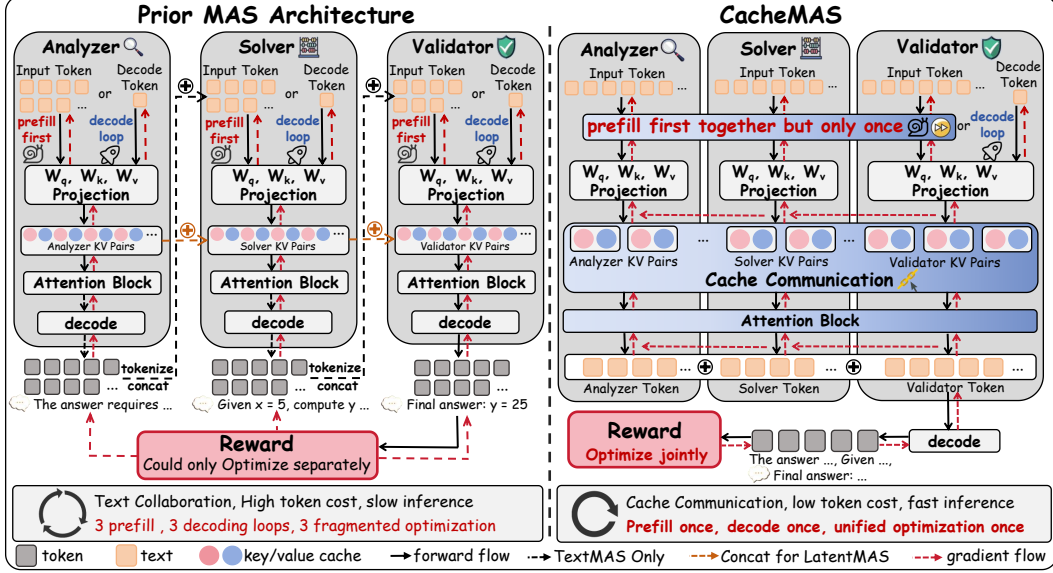


Figure 4: **Prior MAS vs CacheMAS.** **Left:** prior MAS runs a separate prefill-decode pipeline per agent, passing text (TextMAS) or latent KV cache (LatentMAS). Agents could only be optimized separately. **Right:** CacheMAS folds all agents into one prefill via cache communication. Only the Validator decodes, and the single autoregressive rollout admits joint optimization from rewards.

2.3 Toward single-pass cache communication

These findings on untrained latent reasoning imply a clean simplification. Intermediate text outputs are already unnecessary, and we drop the latent loop too, since it adds little inter-agent signal and its states drift outside the regime the model can interpret. The three prefill chains then lose their last serial dependency on sampled or latent tokens and collapse into a single pass with *exactly one prefill and one decode per problem*, sharply cutting per-rollout tokens and prefill wall time. We instantiate this as CacheMAS (Section 3), a single-pass architecture using *cache communication* as the inter-agent interface. Because the pipeline is one autoregressive rollout, standard sequence-level RL (GRPO; Section 3.2) can jointly optimize all agents at once, collapsing per-example training time from multiple per-agent passes down to one.

3 CacheMAS

3.1 Architecture

Figure 4 contrasts the two designs. Acting on the finding of Section 2, CacheMAS uses **only the prefill cache** as the inter-agent communication interface, without text decoding between agents or per-step latent reasoning. We call this *cache communication*, distinguishing it from latent-communication designs that augment the prefill cache with latent reasoning loops. We now formalize CacheMAS.

First, we use three agents (Analyzer, Solver, Validator), indexed by $t \in \{1, 2, 3\}$ and parameterized by three independent sets of weights $\theta = (\theta_1, \theta_2, \theta_3)$ with three role prompts $p = (p_1, p_2, p_3)$. For input problem x , the model input is the concatenation

$$z = [(p_1 \oplus x)_{\text{seg}=1}, (p_2 \oplus x)_{\text{seg}=2}, (p_3 \oplus x)_{\text{seg}=3}], \quad (1)$$

where $\oplus$ denotes concatenation and $(\cdot)_{\text{seg}=k}$ tags each token in its argument with segment id k . CacheMAS runs one prefill over z followed by one decode loop. Within every transformer block, segment- t tokens use θ_t at the projection and feed-forward modules. Cache communication happens at attention layers, which run over tokens with causal masking and let later segments read earlier segments' K, V projections. The per-segment, per-layer K, V projections accumulate into KV cache:

$$C_{\leq 3}(z; \theta_1, \theta_2, \theta_3) = \{(K_{l,t}, V_{l,t})\}_{l \in [L], t \in \{1,2,3\}} \quad (2)$$

where L is the number of transformer layers, $K_{l,t}, V_{l,t}$ are the layer- l K, V projections of segment- t tokens computed under θ_t , and $C_{\leq t}$ is the KV cache slice over segments 1 through t . Decoded tokens append to segment 3, so each new token y_j goes through θ_3 (projection, attention over $C_{\leq 3}$ and prior decoded tokens $y_{<j}$ and feed-forward) and is sampled through θ_3 's language-model head:

$$\pi_\theta(y_j | x, y_{<j}) = \pi_{\theta_3}(y_j | C_{\leq 3}(z; \theta_1, \theta_2, \theta_3), y_{<j}), \quad (3)$$

where z is determined by x via Eq. 1, so $C_{\leq 3}$ implicitly conditions on x . As decode appends new K, V projections to the cache, cache communication continues through the decode loop. The full CacheMAS policy factorizes autoregressively as

$$\pi_\theta(y | x) = \prod_j \pi_\theta(y_j | x, y_{<j}), \quad (4)$$

3.2 End-to-end GRPO

Eq. 4 is a standard autoregressive policy, so training reduces to sequence-level reinforcement learning. We use Group Relative Policy Optimization (GRPO) [Shao et al., 2024]: each input x produces n trajectories $\{y^{(i)}\}$ from the same policy, sharing the deterministic cache $C_{\leq 3}$, and each trajectory gets a standardized advantage $\hat{A}^{(i)}$ by group-normalizing scalar rewards. Rewards are task-specific final-answer scalars (correctness for math and multi-hop QA, unit-test pass rate for code).

GRPO objective. With the importance ratio $\rho^{(i)}$ [Schulman et al., 2017], the per-prompt loss applies clipped policy-gradient together with a per-token KL penalty [Ouyang et al., 2022]:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_x \left[\frac{1}{n} \sum_{i=1}^n \min\left(\rho^{(i)} \hat{A}^{(i)}, \text{clip}(\rho^{(i)}, 1 - \epsilon, 1 + \epsilon) \hat{A}^{(i)}\right) \right] + \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}}), \quad (5)$$

with PPO clipping threshold ϵ , KL weight β and a fixed reference policy π_{ref} .

Gradient structure. Two complementary gradient flows emerge from Eq. 5 on the forward graph of Eq. 4, both visible in Figure 4 (right). The first flow goes from the final-answer reward into all three weight sets. By the chain rule, for the prefill agents $a \in \{1, 2\}$,

$$\nabla_{\theta_a} \log \pi_\theta(y | x) = \frac{\partial \log \pi_\theta(y | x)}{\partial C_{\leq 2}} \cdot \frac{\partial C_{\leq 2}}{\partial \theta_a}, \quad (6)$$

with $C_{\leq 2}$ treated as a sub-tensor of $C_{\leq 3}$ for the chain rule. This gradient is non-zero because $\log \pi_\theta(y | x)$ depends on $C_{\leq 2}$ through every decode step's attention over the cache, and $C_{\leq 2}$ depends on θ_a through projections. The second flow runs between agents through cache communication, in the reverse of agent order. The Validator's gradient signal lands first on θ_2 (via the segment-2 slice of $C_{\leq 2}$), then on θ_1 (via the segment-1 slice). Prior text-MAS and latent-MAS pipelines both insert a per-agent generation loop between consecutive agents, detaching the gradient through token sampling in text-MAS and through impractically deep latent loops in latent-MAS. Cache communication reconnects it. A single GRPO step updates the whole MAS at once without separate agent optimization and per-agent rewards. End-to-end RL also specializes each θ_t for its role, replacing the per-segment prompt as the inductive bias for role assignment.

3.3 Implementation

We implement per-segment projection and feed-forward as wrappers on vLLM [Kwon et al., 2023] and HuggingFace, sharing one implementation for verl [Sheng et al., 2024] training and rollout. Engineering details are in Appendix A.

4 Experiments

4.1 Setup

We compare four configurations. **TextMAS** is the three-agent text-passing baseline. **LatentMAS** [Zou et al., 2025] runs 40 latent reasoning steps per agent, the best setting in our sweep, and passes the

Table 1: Main results across benchmarks. Bold marks the best score in each row per base model.

Task	Dataset	Metric	Qwen3-8B					Hunyuan-1.8B		
			Single-CoT	TextMAS	LatentMAS	CacheMAS	CacheMAS+RL	TextMAS	CacheMAS	CacheMAS+RL
Math	AIME 24	pass@16	83.3	86.7	83.3	90.0	90.0	46.7	50.0	50.0
		avg@16	70.0	72.5	66.9	70.0	74.8	14.2	16.5	16.9
	AIME 25	pass@16	86.7	80.0	81.7	80.0	83.4	40.0	36.7	46.7
		avg@16	61.7	59.2	54.4	56.0	62.7	12.7	14.8	17.3
	AIME 26	pass@16	83.3	80.0	81.7	80.0	83.3	43.3	36.7	43.3
		avg@16	59.6	60.4	56.9	59.2	63.5	10.0	9.8	11.9
Multi-hop QA	HotPotQA	pass@3	42.6	38.9	41.3	40.5	41.2	3.2	25.7	30.7
		avg@3	37.9	31.9	36.6	35.1	37.9	1.1	12.9	19.5
	Bamboogle	pass@3	43.2	44.0	46.0	44.0	46.0	0.8	0.8	6.4
		avg@3	33.9	33.9	37.1	36.8	38.0	0.3	0.3	2.4
	MusiQue	pass@3	47.8	48.1	51.8	53.7	54.3	1.2	19.2	25.5
		avg@3	40.6	39.0	45.4	45.9	47.2	0.4	8.6	13.1
Code	MBPP+	avg@3	78.0	75.9	78.2	77.8	80.4	18.9	52.8	51.4
	HumanEval+	avg@3	89.0	89.2	88.6	89.6	90.2	28.0	64.8	65.7

resulting KV cache to the next agent. **CacheMAS** folds all agents into a single segment-routed forward pass via cache communication (Section 3.1), and **CacheMAS+RL** adds end-to-end GRPO on top (Section 3.2). Base models are Qwen3-8B [Yang et al., 2025] and Hunyuan-1.8B [Hunyuan Team, 2025]. Benchmarks span math (AIME 24/25/26), multi-hop QA (Bamboogle [Press et al., 2023], HotPotQA [Yang et al., 2018], MusiQue [Trivedi et al., 2022]), and code (MBPP+ [Austin et al., 2021] and HumanEval+ [Chen et al., 2021], with max tokens of 20,480 for math, 4,096 for multi-hop QA, and 12,288 for code. We report pass@ N and avg@ N [Chen et al., 2021], with task-specific N shown in Table 1.

For CacheMAS+RL, we train one epoch per domain on the matched train set, DAPO-Math [Yu et al., 2025] for math, APPS [Hendrycks et al., 2021] and CodeContests [Li et al., 2022] for code, and the HotpotQA train split [Yang et al., 2018] for multi-hop QA. Each problem produces $n=8$ GRPO rollouts with all-correct and all-wrong groups filtered out. More details are in Appendix E.

4.2 Main results

Cache communication alone is competitive. With the base model fixed and only the inter-agent interface varying, training-free CacheMAS averages +4.3 avg@3 over TextMAS on multi-hop QA, is at parity on code, and stays within ~ 3 avg@16 on AIME while matching or beating its pass@16. Against LatentMAS, training-free CacheMAS leads on all three AIME years (+2.3 avg@16 on average), MusiQue, and HumanEval+, and trails slightly on HotPotQA, Bamboogle, and MBPP+, while replacing the latent loop with a single prefill.

Joint optimization adds further gains. End-to-end GRPO lifts CacheMAS above TextMAS on every Qwen3-8B benchmark, including AIME, where training-free CacheMAS had been comparable but not above TextMAS. CacheMAS+RL also has the highest avg@ N of all MAS configurations on every benchmark, LatentMAS included. The largest marginal gain from RL over training-free CacheMAS is on AIME (avg +5.3 on avg@16 across the three years, peak +6.7 on AIME 25).

Both findings hold on Hunyuan-1.8B. On Hunyuan-1.8B, training-free CacheMAS sits at parity with TextMAS on math, yet leads it by an unusually large margin on multi-hop QA and code. We checked the rollouts and found that TextMAS outputs on these benchmarks frequently contain garbled tokens and looped repetition that persist into the final agent’s output. Cache communication likely mitigates this because its inter-agent interface is a prefilled KV cache rather than re-decoded text. Similarly to Qwen3-8B, end-to-end RL further lifts CacheMAS above TextMAS on every benchmark, including AIME. CacheMAS+RL beats TextMAS on every AIME year (average +3.1 avg@16).

4.3 Token efficiency and inference speed

Alongside accuracy on every benchmarks, we also record per-problem average tokens, single-rollout time, and prefill time on Qwen3-8B with vLLM (Table 2). Because vLLM cannot ingest latent embeddings, LatentMAS rollouts must use the HF backend, so we omit its single-rollout time as

Table 2: Per-rollout cost on Qwen3-8B. Tokens in thousands. s/rollout is single-rollout time. Prefill is wall-clock seconds per problem. Full breakdown in App. B.

Dataset	TextMAS			LatentMAS		CacheMAS			CacheMAS+RL		
	Tokens	s/rollout	Prefill	Tokens	Prefill	Tokens	s/rollout	Prefill	Tokens	s/rollout	Prefill
AIME 24	39k	5.5	6.60	9.8k	0.37	11.4k	2.4	0.20	12.1k	3.5	0.20
AIME 25	41k	6.0	7.01	10.8k	0.38	12.4k	3.0	0.30	14.9k	4.8	0.19
AIME 26	37k	4.8	6.08	10.6k	0.36	12.0k	2.6	0.20	12.9k	3.8	0.32
HotPotQA	9.8k	1.1	2.26	7.1k	0.56	5.4k	0.45	0.44	5.7k	0.48	0.44
Bamboogle	4.9k	0.74	0.96	1.2k	0.36	1.1k	0.14	0.17	1.1k	0.16	0.22
MusiQue	14k	0.69	2.81	10.5k	0.72	8.7k	0.42	0.58	9.0k	0.46	0.58
MBPP+	12k	1.1	2.24	3.1k	0.37	3.1k	0.35	0.18	3.0k	0.31	0.21
HumanEval+	15k	1.4	2.54	3.7k	0.37	3.6k	0.91	0.28	3.6k	0.90	0.23

not fair and approximate its prefill on vLLM by loading per-stage prompts, a lower bound that excludes the latent embeddings each intermediate stage would also process. Against TextMAS, CacheMAS+RL uses 21%–61% of the tokens, runs $1.5\times$ – $5.3\times$ faster end-to-end, and prefills up to $\sim 33\times$ faster on AIME, Pareto-dominating TextMAS at fixed hardware. Against LatentMAS, CacheMAS roughly matches on tokens overall and is ~ 1.2 – $2.1\times$ faster on prefill even on the lower-bound measurement above. Both speedups stem from cache communication, collapsing MAS’s per-agent prefill-and-decode chains into exactly one prefill and one decode. Hardware and memory are matched across conditions (App. A.5).

5 Ablations and analysis

We address three complementary questions. Section 5.1 asks which components of the multi-agent cache structure the trained pipeline depends on, by perturbing the inter-agent KV cache before and after RL. Section 5.2 asks whether end-to-end joint optimization is what drives CacheMAS+RL’s gains, by comparing against per-agent MAS-GRPO under matched compute. Section 5.3 asks how the cache content itself changes after RL, by measuring per-problem cache alignment with the gold answer and with correct full-rollout trajectories.

5.1 Cache perturbation: training strengthens cache dependence

We perturb the inter-agent cache structure six ways (Table 3: agent removal, cache-content shuffle, execution-order swap, plus the unperturbed baseline) on both training-free CacheMAS and the matched post-RL Qwen3-8B CacheMAS, on AIME 24/25/26 ($n = 3$, 270 rollouts). Before training, only the validator-only mode clearly hurts (-5.6 pp avg@3). The cache-content shuffle, order swap, and single-agent-removal perturbations are all flat or slightly positive. After training, every perturbation costs accuracy. The validator-only mode drops -9.3 pp, and even the smallest perturbations cost ~ 2 pp. All five perturbation deltas shift more negative by -2.2 to -5.6 pp. The trained pipeline is markedly more cache-dependent than the training-free one, supporting that end-to-end RL specializes the three agents so the early agents’ caches carry sample-specific, order-sensitive content the validator depends on.

Table 3: Cache perturbation on AIME 24/25/26. A/S/V = Analyzer/Solver/Validator. Sh = shuffled prefix from a different problem. Sw = swapped A $\leftrightarrow$ S order.

A	S	V	Sh	Sw	CacheMAS		CacheMAS-RL	
					avg@3	pass@3	avg@3	pass@3
✓	✓	✓	×	×	60.4	75.6	66.7	77.8
×	×	✓	×	×	54.8	66.7	57.4	74.4
✓	×	✓	×	×	60.4	75.6	64.4	75.6
×	✓	✓	×	×	61.9	77.8	64.1	77.8
✓	✓	✓	✓	×	61.1	73.3	61.9	80.0
✓	✓	✓	×	✓	61.9	74.5	64.8	75.6

5.2 End-to-end vs per-agent optimization

Prior MAS RL [Liu et al., 2025] optimizes each agent independently because the per-agent generation loop in every prior interface (sampled tokens in TextMAS, deep latent reasoning loop in LatentMAS) breaks gradient flow between agents (Section 3.2). The standard fix is per-agent reward broadcast, replaying the scalar reward to each agent’s decode head and updating once per agent per problem. CacheMAS reconnects the gradient through cache communication, so a single scalar trajectory reward propagates to all three agents under one sequence-level GRPO step. This collapses three per-agent updates per problem into one joint update with no per-role reward design. The remaining question is whether joint optimization is competitive with per-agent on accuracy.

We compare CacheMAS+RL against per-agent MAS-GRPO [Liu et al., 2025] (the standard per-agent GRPO recipe applied to TextMAS, with reward broadcast to each agent’s decode head) on Hunyuan-1.8B multi-hop QA, matched on training recipe (Section 4.1), model, and data. CacheMAS+RL beats MAS-GRPO by +4.0 avg@3 on HotpotQA and +5.1 on MusiQue, with parity on Bamboogle (Figure 5). On the same hardware, CacheMAS+RL evaluates HotpotQA in 2.3 h versus MAS-GRPO’s 15.5 h ($\sim 6.6\times$ wall-time gap), since MAS-GRPO passes text between agents, every agent must decode a full response, producing $\sim 3.7\times$ more tokens per rollout. RL training inherits the same speedup, since rollout collection dominates per-step cost. Joint optimization is therefore both more accurate and substantially faster than per-agent broadcast.

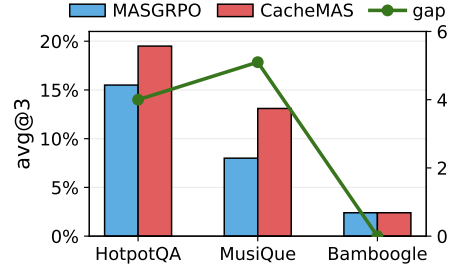


Figure 5: CacheMAS vs MAS-GRPO on Hunyuan-1.8B multi-hop QA at matched compute (avg@3 bars; gap line on right axis).

5.3 What joint optimization changes

To probe what joint optimization changes, we compare CacheMAS and CacheMAS+RL probability assignments on Hunyuan-1.8B with 1000 sampled problems from all benchmarks. For each problem we compute the log-probability of the gold first content token and CacheMAS+RL density curve has mean +1.60 higher than CacheMAS curve (Figure 6 left). We use the first token as the cleanest probe because no prior gold tokens are leaked, while later tokens are already constrained by their predecessors. To check the shift is discriminating rather than uniform, we additionally sample correct and incorrect trajectories from both CacheMAS and CacheMAS-RL, then compute each trajectory’s $\Delta \log P = \log P_{\text{CacheMAS+RL}} - \log P_{\text{CacheMAS}}$. The two density curves are well-separated, with correct trajectories centered +0.65 to the right of zero and incorrect ones -1.28 to the left (Figure 6 right). Joint optimization is therefore a discriminating shift, raising correct outputs and lowering incorrect ones rather than uniformly inflating both. Qwen3-8B yields the same conclusion (Appendix C.2).

Concretely, this distributional shift manifests as identifiable behavioral changes in CacheMAS+RL outputs. We sample 200 problems that CacheMAS+RL got right but CacheMAS got wrong on Qwen3-8B, and inspect their `<think>` blocks. The improvements after RL fall into four patterns: (i) **inferring from indirect evidence** when direct evidence is absent (e.g., concluding “Norman Blake is older than Craig Nicholls” from activity-year cues without explicit birthdates); (ii) **self-correction** when pre-RL reasoning landed on the gold but the final commit drifted; (iii) **cross-document**

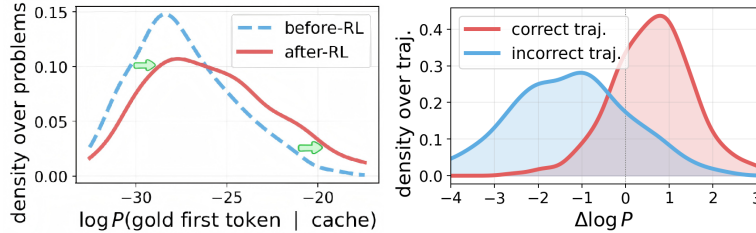


Figure 6: Hunyuan-1.8B CacheMAS-RL probability shifts on sampled problems. **Left:** log-probability of gold first content token. **Right:** per-rollout $\Delta \log P$ for correct vs incorrect trajectories.

integration (e.g., expanding “SFC” to “Sergeant First Class” via a second document); and (iv) **using world knowledge** not in the documents. Full case studies are in Appendix C.4.

6 Related Work

Text-based LLM MAS. MAS built on LLMs have been studied as inter-agent debate [Du et al., 2023], role-played planner-executor and software-engineering pipelines [Wu et al., 2024, Hong et al., 2024], self-reflection and verbal reinforcement [Shinn et al., 2023], and tool-using reasoning agents [Yao et al., 2023]. They differ in role decomposition and coordination protocol but share one interface assumption, namely that each agent’s contribution is materialised as text, decoded by one agent and re-encoded by the next via a fresh prefill pass. CacheMAS does not propose a new role decomposition. It targets this interface assumption, replacing the text channel with KV cache as the inter-agent communication interface, produced in one segment-routed forward pass.

Latent reasoning and latent communication. Two existing lines motivate non-text channels for LLM computation. *Latent reasoning* performs intermediate computation in a continuous latent space rather than emitting discrete reasoning tokens, instantiated as continuous-thought training [Hao et al., 2025], recurrent-depth latent computation [Geiping et al., 2025], and continuous-concept-space sampling [Zhang et al., 2025], with Chen et al. [2025] providing a recent survey of the line. *Latent communication* replaces inter-LLM text exchange with direct KV-cache transfer, either through a learned semantic projection [Fu et al., 2025] or via training-free cross-context cache reuse for multi-agent serving [Ye et al., 2025]. Both ideas extend to multi-agent collaboration. LatentMAS [Zou et al., 2025] replaces inter-agent text decoding with autoregressive latent thought generation through a shared latent working memory, and Thought Communication [Zheng et al., 2025] formalises the inter-agent latent exchange as a latent variable model with identifiability guarantees for shared and private thoughts. We find the prefill cache carries the inter-agent signal while per-step latent transitions contribute little, so CacheMAS drops the latent loop, producing cache within one segment-routed prefill rather than transferring it across separate latent reasoning passes.

LLM RL and MAS RL. Recent reasoning-focused systems use sequence-level RL with verifiable final-answer rewards, including GRPO [Shao et al., 2024], DeepSeek-R1 [Guo et al., 2025], and DAPO [Yu et al., 2025], with process reward models that verify intermediate steps as a complementary line [Khalifa et al., 2025]. These objectives apply directly to a single autoregressive trajectory. Lifting them to MAS by simply broadcasting the final-answer reward to each agent’s decode head is coarse, since the reward says nothing about which agent’s contribution moved the trajectory. Specialised MAS-RL methods address this gap along four axes. *Reward shaping* learns dense per-step signals so that smaller LLMs reach competitive agentic-RL performance [Zhu et al., 2025]. *Turn-level credit attribution* scores each agent turn independently with an LLM judge [Tan et al., 2025] or decouples a meta-thinker and an executor with turn-level optimisation [Wan et al., 2025]. *Role-level optimisation* broadcasts a shared scalar reward to per-agent decode heads [Liu et al., 2025, Feng et al., 2026] or uses hierarchical group-relative advantages for main and sub-agents [Hong et al., 2025]. *Path or tree-search-based credit assignment* constructs tree-structured rollouts so that step-wise process supervision can be inferred from outcome-only rewards [Ji et al., 2026]. By collapsing the pipeline into a single autoregressive rollout, CacheMAS uses vanilla GRPO [Shao et al., 2024] unchanged, sidestepping the per-agent credit assignment and these specialised multi-agent RL machinery.

7 Conclusion

CacheMAS reframes MAS via cache communication. Our controlled study shows that the prefill-pass KV cache alone, not per-step latent transitions, carries the dominant inter-agent signal, motivating a single-pass design. Three role-specialized agents run a single segment-routed prefill that produces KV cache as communication interface, collapsing the staged MAS pipeline into a single autoregressive rollout that sequence-level RL optimizes jointly from final-answer rewards, without per-agent reward design or specialized multi-agent RL machinery. Without training, CacheMAS already uses about $3\times$ fewer tokens and runs $2\times$ faster than TextMAS, matching or exceeding it on multi-hop QA and code and staying on par with LatentMAS. Joint optimization then lifts CacheMAS above TextMAS on math, multi-hop QA, and code on both Qwen3-8B and Hunyuan-1.8B, with our analyses tracing the gain to one rollout-level reward propagating through cache communication. Cache communication

also makes RL training about $6\times$ faster than per-agent MAS-GRPO recipes. Admittedly, our work has limitations: we only evaluated latent reasoning without training. We will explore a jointly optimized MAS framework incorporating trainable latent reasoning in the future.

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- X. Chen, A. Zhao, H. Xia, X. Lu, H. Wang, Y. Chen, W. Zhang, J. Wang, W. Li, and X. Shen. Reasoning beyond language: A comprehensive survey on latent chain-of-thought reasoning. *arXiv preprint arXiv:2505.16782*, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- Lang Feng, Longtao Zheng, Shuo He, Fuxiang Zhang, and Bo An. Dr. MAS: Stable reinforcement learning for multi-agent LLM systems. *arXiv preprint arXiv:2602.08847*, 2026.
- Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Conference on Language Modeling (COLM)*, 2025.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- Haoyang Hong et al. Multi-agent deep research: Training multi-agent systems with M-GRPO. *arXiv preprint arXiv:2511.13288*, 2025.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

- Hunyuan Team. Hunyuan-1.8B-Instruct. <https://huggingface.co/tencent/Hunyuan-1.8B-Instruct>, 2025. Hunyuan dense series, supports fast/slow thinking modes and 256K context.
- Yuxiang Ji et al. Tree search for LLM agent reinforcement learning. *arXiv preprint arXiv:2509.21240*, 2026. ICLR 2026.
- Muhammad Khalifa et al. Process reward models that think. *arXiv preprint arXiv:2504.16828*, 2025.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Andrei Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Shuo Liu, Tianle Chen, Zeyu Liang, Xueguang Lyu, and Christopher Amato. LLM collaboration with multi-agent reinforcement learning. *arXiv preprint arXiv:2508.04652*, 2025.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Weiting Tan et al. Process-supervised reinforcement learning for interactive multimodal tool-use agents. *arXiv preprint arXiv:2509.14480*, 2025.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.
- Ziyu Wan et al. ReMA: Learning to meta-think for LLMs with multi-agent reinforcement learning. *arXiv preprint arXiv:2503.09501*, 2025.

- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. KVCOMM: Online cross-context KV-cache communication for efficient LLM-based multi-agent systems. *arXiv preprint arXiv:2510.12872*, 2025. NeurIPS 2025.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, Shuohang Wang, Yelong Shen, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space. *arXiv preprint arXiv:2505.15778*, 2025.
- Yujia Zheng, Zhuokai Zhao, Zijian Li, Yaqi Xie, Mingze Gao, Lizhu Zhang, and Kun Zhang. Thought communication in multiagent collaboration. *arXiv preprint arXiv:2510.20733*, 2025.
- Siyu Zhu et al. Planner-R1: Reward shaping enables efficient agentic RL with smaller LLMs. *arXiv preprint arXiv:2509.25779*, 2025.
- Jiaru Zou, Xiyuan Yang, Ruizhong Qiu, Gaotang Li, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 accurately summarize the contributions developed in Sections 2–5, including the controlled-study finding (prefill cache as dominant inter-agent signal in untrained latent MAS), the CacheMAS architecture, joint optimization via vanilla GRPO, the empirical efficiency and accuracy results, and the limitation that latent reasoning was evaluated only without training.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 explicitly states the scope limitation that latent reasoning was evaluated only without training and that jointly training latent reasoning with MAS RL is left to future work. Section 2 also scopes the prefill-cache-dominance finding to the untrained latent MAS setting.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper presents no formal theorems requiring proofs. The gradient-flow argument in Section 3.2 (Eq. 6) is a chain-rule exposition of why GRPO updates propagate to all three weight copies through cache communication, not a theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The CacheMAS architecture and segment-routed forward pass are described in Section 3 with full engineering details (including the `MultiCopyLinear` reference implementation, vLLM and verl integration) in Appendix A. Training recipes, hyperparameters, datasets, and hardware are documented in Section 4.1 and Appendices E–A.5. All benchmarks are public.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Source for the MultiCopyLinear module, the HuggingFace multi-agent wrapper, the vLLM model-runner wrapper, and the verl integration scripts will be released alongside the paper (Appendix A.6). Per-domain training and rollout configurations are reproduced in Appendix E, and all benchmarks (AIME, HotpotQA, Bamboogle, MusiQue, MBPP+, HumanEval+) are publicly available.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 states the four configurations, base models, benchmarks, decoding budgets, training data, and rollout protocol. Appendix E lists the full per-domain GRPO hyperparameters, optimizer settings, and per-condition prompt structures, and Appendix A.5 states hardware and KV-cache budgets.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not report per-cell error bars. Each main-table accuracy aggregates over many sample-level evaluations: AIME pass@16 and avg@16 use 16 rollouts per problem $\times$ 30 problems per year, multi-hop QA pass@3 and avg@3 use 3 rollouts per problem $\times$ thousands of problems, and the Section 5.1 perturbation study reports $n=3$ rollouts $\times$ 90 AIME problems (270 rollouts per cell). Re-running every cell with multiple training seeds was beyond our compute budget.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix A.5 lists the GPU configuration for each condition ($4 \times$ A100-PCIE-40GB for multi-hop QA on both base models, $4 \times$ A100-80GB for Qwen3-8B math/code, $4 \times$ A100-80GB for LatentMAS rollouts). Per-rollout wall-clock time and prefill time are reported in Table 2 and Section 5.2 (CacheMAS+RL HotpotQA in 2.3 h vs MAS-GRPO’s 15.5 h on the same hardware). Per-domain training wall-clock time is documented in Appendix E.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This work uses publicly released base models and standard reasoning benchmarks, involves no human subjects or personally identifiable data, and does not introduce models that pose new misuse risks.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The work is foundational research on multi-agent LLM efficiency and joint optimization, with no direct path to a particular application or deployment. Cache communication primarily reduces inference cost of existing reasoning pipelines and does not introduce new dual-use capabilities beyond those already enabled by the underlying base models.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases code and trained adapters on top of publicly released base models; no new high-misuse-risk asset (e.g., new pre-trained foundation model, image generator, or scraped dataset) is introduced.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All base models (Qwen3-8B [Yang et al., 2025], Hunyuan-1.8B [Hunyuan Team, 2025]) and benchmarks (AIME, HotpotQA [Yang et al., 2018], Bamboogle [Press et al., 2023], MusiQue [Trivedi et al., 2022], MBPP+/HumanEval+ [Austin et al., 2021, Chen et al., 2021, Liu et al., 2023]) and training corpora (DAPO-Math [Yu et al., 2025], APPS [Hendrycks et al., 2021], CodeContests [Li et al., 2022]) are cited at first use and used in accordance with their respective public licenses.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The new asset is the CacheMAS implementation (Appendix A, including the MultiCopyLinear module, vLLM model-runner wrapper, HuggingFace multi-agent wrapper, and verl integration). The release will include per-domain training and rollout configurations from Appendix E as documentation.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work involves no crowdsourcing and no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This work involves no human subjects, so no IRB approval is required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are the object of study (CacheMAS is an architecture over base LLMs evaluated as such), not a non-standard component of the research methodology itself. Any LLM use during writing was limited to editing and formatting.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Implementation details

This appendix expands Section 3.3 with the engineering details of the segment-routed forward pass, the vLLM rollout-time routing wrapper, the KV-cache layout, and the verl integration used for end-to-end GRPO.

A.1 Segment-routed forward on the training side

The three weight copies $\theta = (\theta_1, \theta_2, \theta_3)$ are held in GPU memory as three independent parameter sets of the base model. We patch every per-block projection module (Q, K, V, output, and the FFN gate / up / down projections) to a thin `MultiCopyLinear` wrapper that stacks the three copies along an extra leading axis and dispatches each token to its segment’s copy via a per-token `segment_id` buffer. The buffer is filled at the input layer once per batch and propagated through the forward pass alongside the hidden states. Within `MultiCopyLinear` we partition the input tensor by `segment_id`, multiply each partition by its segment’s weight slice, and scatter the outputs back into the original token order. Backward through this scatter is the autograd source of Eq. 6: gradients accumulate into each θ_k slice independently because the partition is exact and the scatter is a no-op on the chain rule. The attention operator itself is untouched. The Q / K / V tensors emitted by `MultiCopyLinear` flow into standard scaled-dot-product attention with the regular causal mask, and the resulting KV pairs are appended to the shared cache in their native segment order.

The reference `MultiCopyLinear` forward is a ~ 15 -line PyTorch module:

```
class MultiCopyLinear(nn.Module):
    """Per-segment routing: each token uses its segment's weight slice."""
    def __init__(self, in_dim, out_dim, n_segments=3):
        super().__init__()
        # weight: (n_segments, in_dim, out_dim) -- three independent copies
        self.weight = nn.Parameter(torch.empty(n_segments, in_dim, out_dim))
        self.n_segments = n_segments

    def forward(self, x, segment_id):
        # x: (... , in_dim); segment_id: (...) per-token int in [0, n_segments)
        out = torch.empty(*x.shape[:-1], self.weight.shape[-1],
                          dtype=x.dtype, device=x.device)
        for k in range(self.n_segments):
            mask = (segment_id == k)
            if mask.any():
                out[mask] = x[mask] @ self.weight[k]
        return out
        # autograd: d(out)/d(self.weight[k]) is non-zero only on tokens with
        # segment_id == k, so each theta_k slice receives an independent
        # gradient, matching the chain rule in Eq.~\ref{eq:gradient}.
```

A.2 Rollout-time routing in vLLM

For inference we add a thin model-runner wrapper to vLLM that mirrors the training-side routing. The wrapper carries `segment_id` alongside `token_id` through vLLM’s request scheduling and substitutes the same `MultiCopyLinear` dispatch into vLLM’s projection-layer calls. PagedAttention [Kwon et al., 2023] is unmodified. The cache pool sees a single sequence of tokens per request with the standard slot mapping, and per-segment routing happens entirely inside the projection layer. Decoded tokens are unconditionally appended to segment 3 (the Validator), so under the routing rule each new token runs through θ_3 at every block and is sampled through θ_3 ’s language-model head, matching Eq. 3. Continuous batching and prefix sharing both work as in the unmodified vLLM stack.

A.3 KV-cache layout and attention kernels

The shared KV cache is held in vLLM’s standard paged form. Each transformer block produces one set of KV pairs per token, stored in the paged pool keyed by the request’s virtual block table. Because

attention is a single causal pass over all segments, the cache-pool sizing rule is identical to a single-agent vLLM deployment of the same base model at the same context length. FlashAttention [Dao et al., 2022] kernels are reused as-is, with no segment-id conditioning inside the kernel. The only departure from a vanilla vLLM serving stack is the `MultiCopyLinear` dispatch in the projection layer, which adds three small tensor-contiguity gathers per block but does not introduce extra global synchronization.

A.4 verl integration and reference policy

End-to-end GRPO uses verl [Sheng et al., 2024] as the outer training loop. We add a HuggingFace-side multi-agent model wrapper that exposes a single `forward` and `log_prob` interface to verl’s GRPO trainer while internally routing each token to its segment’s weight copy. Rollout queries are served by the vLLM wrapper above. The KL-regularization reference policy π_{ref} is a frozen clone of the same multi-copy stack at initialization, so the per-token KL is computed under the same routing as the trained policy. AdamW optimizer state is held per-copy with no cross-copy sharing.

A.5 Hardware and parallelism

Both base models train and roll out on 4-GPU nodes with tensor parallelism $\text{TP}=4$. Qwen3-8B math and code use $4 \times \text{A100-80GB}$; Qwen3-8B and Hunyuan-1.8B multi-hop QA use $4 \times \text{A100-PCIE-40GB}$. LatentMAS rollouts uniformly use $4 \times \text{A100-80GB}$ across all benchmarks because its HF-backend latent loop is too slow on the smaller PCIE-40GB nodes. The three-copy parameter footprint is the main extra storage cost over a single-agent baseline. The KV-cache pool size is unchanged from the single-agent vLLM default for the same context-length budget, which yields equal per-token KV memory across CacheMAS and TextMAS conditions regardless of agent count. The TextMAS baseline uses our HF-side multi-agent wrapper so that tokenizer template, decoding parameters, and stop criteria match the CacheMAS path. Per-domain wall-clock times and full GRPO hyperparameters are listed in Appendix E.

A.6 Reproducibility

Source for the `MultiCopyLinear` module, the HuggingFace multi-agent wrapper, the vLLM model-runner wrapper, and the verl integration scripts will be released alongside the paper. Per-domain training scripts and rollout configurations are reproduced in Appendix E.

B Detailed efficiency tables

The main paper reports a compact summary of token cost, end-to-end wall time, and per-problem prefill on Qwen3-8B for TextMAS vs CacheMAS+RL (Table 2). This appendix provides the full numerical breakdown for both base models, including the training-free CacheMAS row and the LatentMAS-style *Pre* ($3\times$) prefill column on Qwen3-8B.

Setup. We use the same three role prompts, the same hardware ($4 \times \text{A100-PCIE-40GB}$, $\text{TP}=4$, vLLM continuous batching), and the same per-request synchronous prefill-timing methodology for both base models. The Hunyuan-1.8B Pre values were obtained by replaying the saved analyzer/solver outputs from the original Hunyuan eval onto the same hardware as the Qwen3-8B numbers. The doc-once prompt structure used for Qwen3-8B multi-hop QA prefill was not back-applied to Hunyuan, so Hunyuan multi-hop QA Pre values reflect the default doc-thrice prompt structure.

Findings on Hunyuan-1.8B. CacheMAS uses 36–59% of TextMAS tokens across math and multi-hop QA on Hunyuan-1.8B, mirroring the Qwen3-8B reduction range of 21–61%. End-to-end wall-time speedup is $1.0\text{--}3.6\times$ ($\text{TextMAS}/\text{CacheMAS s/roll}$), again similar in shape to Qwen3-8B. Per-problem prefill is $\sim 15\text{--}27\times$ smaller for CacheMAS than TextMAS on AIME (0.11–0.13s vs $\sim 2.9\text{--}3.3\text{s}$, comparable to the $33\times$ ratio at the Qwen3-8B scale) and $\sim 1.5\text{--}2.6\times$ smaller on multi-hop QA. CacheMAS+RL adds 0–22% tokens over training-free CacheMAS with negligible prefill overhead, matching the Qwen3-8B pattern.

The Hunyuan-1.8B accuracy table across all eight benchmarks is deferred to follow-up work.

Table 4: Per-rollout token cost, end-to-end wall time, and per-problem prefill wall time on Qwen3-8B. *Tok* is mean total tokens per rollout. *s/roll* is end-to-end seconds per rollout. *Pre (1×)* is prefill wall-clock per problem when the three role prompts are submitted as a single concatenated 3-segment vLLM request, the deployment pattern of CacheMAS. *Pre (3×)* is prefill wall-clock per problem when the same three role prompts are submitted as three separate vLLM requests with no cross-segment attention, the LatentMAS-style execution pattern. This column is measured on the same multi-agent vLLM model used by CacheMAS (3-agent stacked weights, identical KV-cache pool budget), so the only difference vs the CacheMAS *Pre (1×)* is whether the role prompts are merged into one request or submitted as three. For TextMAS, *Pre (1×)* runs on the standard single-agent vLLM model and is the sum of three prefills, each conditioned on the actual decoded text from the prior agent. All numbers under 4×A100-PCIE-40GB, TP= 4, vLLM continuous batching, per-request synchronous timing. Hunyuan-1.8B numbers and full standard deviations are in Appendix B.

Task	Dataset	TextMAS			CacheMAS			CacheMAS+RL			LatentMAS-style
		Tok	s/roll	Pre (1×)	Tok	s/roll	Pre (1×)	Tok	s/roll	Pre (1×)	Pre (3×)
Math	AIME 24	39.0k	5.5	6.60	11.4k	2.4	0.20	12.1k	3.5	0.20	0.37
	AIME 25	41.3k	6.0	7.01	12.4k	3.0	0.30	14.9k	4.8	0.19	0.38
	AIME 26	36.5k	4.8	6.08	12.0k	2.6	0.20	12.9k	3.8	0.32	0.36
Multi-hop QA	HotPotQA	9.8k	1.1	2.26	5.4k	0.45	0.44	5.7k	0.48	0.44	0.56
	Bamboogle	4.9k	0.74	0.96	1.1k	0.14	0.17	1.1k	0.16	0.22	0.36
	MusiQue	14.2k	0.69	2.81	8.7k	0.42	0.58	9.0k	0.46	0.58	0.72
Code	MBPP+	12.1k	1.1	2.24	3.1k	0.35	0.18	3.0k	0.31	0.21	0.37
	HumanEval+	14.6k	1.4	2.54	3.6k	0.91	0.28	3.6k	0.90	0.23	0.37

Table 5: Per-rollout token cost, end-to-end wall time, and per-problem prefill wall time on Hunyuan-1.8B, complementing the Qwen3-8B numbers in Table 4. Same hardware (4×A100-PCIE-40GB, TP= 4, vLLM continuous batching) and same per-request synchronous prefill timing methodology as the main table. The qualitative ordering between methods is preserved. CacheMAS uses 36–59% of TextMAS tokens with prefill cost $\sim 15\text{--}27\times$ smaller on math and $\sim 1.5\text{--}2.6\times$ smaller on multi-hop QA.

Task	Dataset	TextMAS			CacheMAS			CacheMAS+RL		
		Tok	s/roll	Pre	Tok	s/roll	Pre	Tok	s/roll	Pre
Math	AIME 24	39.3k	3.2	2.93	14.4k	3.0	0.11	14.2k	3.0	0.27
	AIME 25	37.9k	3.1	2.84	14.7k	3.1	0.12	15.7k	3.3	0.15
	AIME 26	43.6k	3.5	3.25	16.8k	3.6	0.13	16.4k	3.4	0.13
Multi-hop QA	HotPotQA	9.2k	2.1	1.11	5.2k	0.38	0.57	5.2k	0.38	0.54
	Bamboogle	7.9k	1.2	0.49	906	0.34	0.09	877	0.35	0.06
	MusiQue	10.4k	3.2	1.24	8.3k	0.45	0.80	8.3k	0.45	0.80
Code	MBPP+	17.4k	6.3	0.94	3.8k	3.2	0.15	3.7k	3.7	0.15
	HumanEval+	19.9k	7.7	1.05	4.5k	4.9	0.14	4.3k	5.3	0.14

C Cache alignment on flipped problems: full evidence

This appendix collects the full quantitative evidence underlying §5.3: per-(model, benchmark) flip census, both probes (*v2* first-token, *v3* trajectory) on both base models, the per-problem mechanism breakdown that the main-text caveat paragraph summarizes, sample case studies, and the prior aggregate-level diagnostics.

C.1 Flip census across base models and benchmarks

For each (model, benchmark) pair we count problems by their before-RL/after-RL majority label transition (with $n = 3$ rollouts in both states; truncated rollouts excluded). Soft-flip is the main-text definition (majority label crosses 0.5); reverse-flip is the same in the opposite direction; strict-flip is the more conservative subset of soft-flip with before-RL 0/3 correct and after-RL $\geq 2/3$.

model	benchmark	n_{prob}	both-correct	both-incorrect	soft-flip	reverse-flip	strict $\subset$ soft	partial-improve
Qwen3-8B	HotpotQA	7405	33.3%	61.4%	3.6%	1.7%	92	339
Qwen3-8B	MusiQue	2417	43.8%	50.4%	2.8%	3.1%	17	140
Hunyuan-1.8B	HotpotQA	7405	7.7%	77.9%	11.5%	2.9%	303	629
Hunyuan-1.8B	MusiQue	2417	3.2%	86.6%	7.7%	2.4%	96	240
Hunyuan-1.8B	AIME 2024	30	13.3%	73.3%	10.0%	3.3%	1	6

Table 6: Per-problem flip census on Qwen3-8B and Hunyuan-1.8B (multi-hop QA + Hunyuan AIME 2024). The soft-flip column is the analysis subset for §5.3; reverse-flip is the analysis subset for Appendix C.5. AIME 2025/2026 and Bamboogle are omitted from analysis (too few flips). The Qwen3 multi-hop QA gain (+1.8 pp on HotpotQA, −0.1 pp on MusiQue) decomposes into soft-flip − reverse-flip + partial-improve contributions consistent with the measured net change.

C.2 Cache-alignment probes on both base models

Figure 7 shows the four-panel (model $\times$ probe) version of §5.3’s main figure: $v2$ gold first-token log-probability and $v3$ paired full-rollout $\Delta \log P$, on Qwen3-8B (HotpotQA) and Hunyuan-1.8B (HotpotQA + MusiQue pooled). The Qwen3 panels carry the smaller-but-format-cleaner signal; the Hunyuan panels carry the larger-but-format-confounded signal as discussed in §5.3.

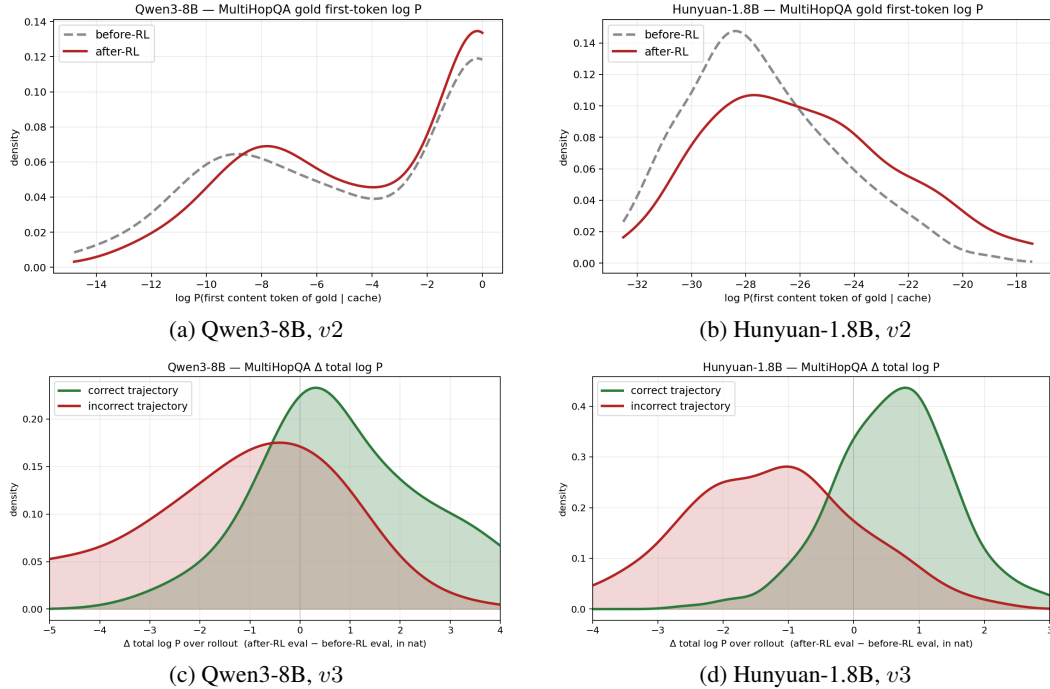


Figure 7: Cache-alignment probes on flipped problems for both base models. (a,b) $v2$: distribution of $\log P(\text{first content token of gold} | \text{cache})$ before vs after RL, on the soft-flipped subset only. Both models show a consistent rightward shift after RL; Hunyuan’s shift is larger (Δ mean +1.60 vs Qwen3’s +0.57 nat). (c,d) $v3$: distribution of paired $\Delta \log P$ over full rollouts, separating known-correct trajectories (green; sampled from after-RL successful rollouts) from known-incorrect ones (red; sampled from before-RL failed rollouts). On both models post-RL eval raises the trajectory log-probability of correct rollouts and lowers it for incorrect ones; the gap is highly significant (MWU $p \leq 3.5e-35$).

C.3 Mechanism breakdown: Qwen3-8B forward-flips

To check whether the cache shift in §5.3 reflects “RL teaches MAS reasoning” or some subset of more specific behaviors, we manually inspect all 267 Qwen3-8B HotpotQA forward-flips. For each

we extract the before-RL incorrect prediction, the after-RL correct prediction, and a snippet of each `<think>` block, and apply a simple decision tree.

mechanism	share	description
B. trim verbose	50%	post-RL prediction is a prefix/suffix/substring of pre-RL (“ <i>Kevin Smith and Scott Mosier</i> ” → “ <i>Kevin Smith</i> ”)
A. refusal → commit	23%	pre-RL was an explicit refusal phrase (“ <i>cannot determine</i> ”); post-RL committed an answer
A'. self-doubt reversal	7%	pre-RL <code><think></code> contained gold answer but final commit went elsewhere; post-RL kept the right commit
yes/no commit	4%	pre-RL gave a verbose hedging explanation; post-RL gave only “Yes”/“No” (subset of B)
format-match adjustment	2%	post-RL prediction matches gold’s surface form more precisely (“9” → “ <i>nine vertical feet</i> ”)
genuine cross-boundary integration	~ 6%	post-RL invokes external knowledge, cross-document triangulation, or question-stem precision
↪ world knowledge	~ 2%	e.g., knows publication date of <i>The Wind in the Willows</i> not in documents
↪ cross-doc evidence	~ 2%	e.g., Sergeant Shughart case: post-RL expands “SFC” via second document
↪ question-stem precision	~ 2%	e.g., answers the predicate the question literally asks for, not a related concept
misc / unclassified	~ 8%	—

Table 7: Mechanism breakdown of all 267 Qwen3-8B HotpotQA forward-flips. Categories are not perfectly disjoint (a self-doubt reversal can also be brief enough to qualify as a trim); we resolve overlaps in the order listed above. The numerical shares are intended as honest order-of-magnitude estimates rather than disjoint partitions; auto-classification heuristics and full per-case breakdown are released alongside the source code.

The numbers tell a layered story. First, ~ 73% of forward-flips ($B + A$) are explainable as direct optimization on the exact-match outcome reward: making correct answers more concise (matching the gold form precisely), or converting epistemic abstention into a committed guess. Second, the remaining ~ 13% ($A' + \text{yes/no} + \text{format-match} + \text{misc}$) is a softer variant of the same: the model already had the gold answer somewhere in its think trace but did not commit to it before RL. Third, only the ~ 6% “genuine cross-boundary integration” bucket reflects RL teaching the model to do things its pre-RL self could not: pulling in world knowledge, triangulating across documents, or matching the exact predicate the question asks. In absolute terms this is ~ 16 problems out of 7405, contributing ~ 0.2 pp of the measured +1.8 pp Qwen3-8B HotpotQA gain. The cache-level signal in §5.3 aggregates all these mechanisms; we caution the reader against reading the cache shift as evidence of richer multi-agent reasoning structure.

C.4 Sample case studies

We include three short before-RL / after-RL excerpts to illustrate the mechanism categories. Full `<think>` blocks for these and additional cases are released with the source code.

Pattern A (refusal → commit), longer-fix. *Question:* Who is older, Craig Nicholls or Norman Blake? *Gold:* Norman Blake. *Before-RL final:* “Cannot determine from the provided documents” (no birth date for either in the documents). *After-RL final:* “Norman Blake”, justified by an inference from activity-year evidence (Norman Blake’s earliest album in 1976 versus The Vines forming in 1994). The reasoning content is the same in the two passes; the difference is whether the model commits to a guess when direct evidence is absent.

Pattern E (cross-document evidence integration). *Question:* Randy Shughart’s rank when he died. *Gold:* Sergeant First Class. *Documents:* “In the Company of Heroes” refers to “SFC Randy Shughart”; the [Gary Gordon] document additionally mentions “Sergeant First Class Randy Shughart” verbatim. *Before-RL final:* “Staff Sergeant” (incorrect expansion of “SFC” from the first document

only). *After-RL final*: “Sergeant First Class” (post-RL cross-references the second document to expand the abbreviation correctly).

Pattern B-reverse (over-trim, reverse-flip honesty). *Question*: which company manufactures one in three objects with a 1–5-year shelf life. *Gold*: Crown Holdings Incorporated. *Before-RL final*: “Crown Holdings Incorporated” (correct, exact match). *After-RL final*: “Crown Holdings” (incorrect by exact-match scoring; the reasoning content is the same and would also satisfy a fuzzy match). This is the dual of Pattern B: post-RL trimming improves exact-match agreement on average but loses problems where the gold answer happens to include a corporate suffix.

C.5 Reverse-flips on Qwen3-8B HotpotQA

Of the 128 Qwen3-8B HotpotQA reverse-flips (problems RL broke), the same auto-classification gives roughly 52% confidently-different content (the model commits to a wrong alternative entity), 16% post-RL added words/garbled output, 13% post-RL induced refusal (“insufficient information” where pre-RL had answered), and 13% over-trim (the dual of Pattern B; the Crown Holdings example above). Net forward – reverse $\approx 267 - 128 = 139$ flipped problems out of 7405 matches the measured Qwen3-8B HotpotQA accuracy gain (≈ 1.9 pp; observed +1.8 pp). RL’s mechanism is therefore not surgical: it changes commitment style globally, helps more often than it hurts on this benchmark, but is far from a clean “RL fixes broken reasoning” picture.

D Aggregate-level representation diagnostics

Prior to the per-problem analysis of §5.3 we ran an aggregate-level representation analysis on a fixed sample of 90 AIME problems and a matched seed-42 sample of 90 HotpotQA problems, applying linear-probe AUROC, effective-rank, wrap-test PCA, and cross-fitted single-direction projection probes. Those probes return null-to-small effects — consistent with the conclusion of §5.3 that the per-flipped-problem subset is the correct grain at which RL’s effect is detectable. We retain the aggregate diagnostics here for completeness.

Setup. For each of the 90 AIME 24/25/26 problems and a fixed seed-42 random sample of 90 HotpotQA test problems (used here only for matched-size representation analysis), we collect the final-layer hidden state $x \in \mathbb{R}^{4096}$ at three positions of the segment-routed forward pass — Analyzer end, Solver end, Validator first-decoded position — on both the training-free and CacheMAS+RL pipelines, with matched prompts. Correctness labels come from the corresponding pipeline’s $n = 3$ rollouts, with a problem labeled positive when at least half are correct. We apply three probes. **Decodability:** AUROC of a logistic probe predicting final-answer correctness, read in two ways (model-matched and same-label, defined below). **Expansion:** effective rank of the cache covariance at 95% explained variance. **Shift:** centroid translation $\|\mu_a - \mu_b\|$ and variance ratio σ_a^2/σ_b^2 in a leading-PC plane fit on the before-RL cloud only. *Alignment* is the operational claim that survives all three: a small same-label-controlled decodability gain coexists with null expansion, null shift, and (as we show below) a null cross-fitted per-problem class-conditional projection. We do not claim the trichotomy is exhaustive — covariance rotations at fixed rank, low-variance directions, and nonlinear changes are not constrained by these specific probes.

Probe AUROC: model-matched vs same-label. We fit a logistic probe $\hat{p}(x) = \sigma(\langle w, x \rangle + b)$ with feature standardization and 5-fold StratifiedKfold + L2-tuned regularization (Table 8). *Model-matched probing* reports AUROC for predicting y_{before} from x_{before} and y_{after} from x_{after} ; this is the headline. The after-minus-before gap is +10–13 pp on AIME and +4–8 pp on HotpotQA. *Same-label probing* holds the prediction target at y_{after} and refits the probe on x_{before} vs x_{after} separately. The same-label gap is –0.6 pp at AIME Analyzer-end (null), +6.3 pp at AIME Solver-end, +7.1 pp at AIME Validator-first, and +1.4 to +3.3 pp across the three HotpotQA positions. Locating the controlled component of the linear-decodability change requires the same-label test: most of AIME’s headline Analyzer-end gain is accounted for by probe refitting and label difficulty rather than by RL-induced cache content the same-label probe can pick up, while AIME Solver/Validator retain ~6–7 pp of controlled gain. (We caution that even the same-label probe still re-fits w on x_{after} , so the gap reflects *relative linear separability under independently-fit probes* rather than the strength of any

Table 8: Probe AUROC for predicting final-answer correctness from cache states. *Model-matched*: each model is probed with its own correctness label (y_{before} for x_{before} , y_{after} for x_{after}); the after-minus-before gap is the headline. *Same-label gap*: prediction target y_{after} is held fixed and the probe is refit on x_{before} and x_{after} separately (in pp). The same-label gap localizes the RL-induced increase in linear decodability; the headline gap conflates this with probe refitting and label-difficulty effects. AIME: 90 AIME 24/25/26 problems. HotpotQA: 90-question random sample (seed 42) from the test split, used here only for matched-size representation analysis (full-split numbers in Table 1). Probes use 5-fold StratifiedKFold + StandardScaler + L2-tuned logistic regression.

Position	AIME (90 problems)			HotpotQA (90-q sample)		
	Model-matched		Same-label gap (pp)	Model-matched		Same-label gap (pp)
	before	after		before	after	
Analyzer-end	0.589	0.722	−0.6	0.641	0.683	+3.3
Solver-end	0.589	0.694	+6.3	0.534	0.615	+1.4
Validator-first	0.611	0.686	+7.1	0.552	0.602	+1.7

fixed direction.) We use the same-label gap as the conservative measure of cache decodability change throughout the rest of the section.

Effective rank: no expansion. We compute the covariance spectrum of cache states across the 90 AIME problems at each probed position and report the number of leading principal components needed to capture 95% of the variance. The before-RL and after-RL effective ranks are nearly identical: 54 vs 54 at Analyzer-end, 57 vs 57 at Solver-end, 58 vs 59 at Validator-first, and $61 \rightarrow 60$, $64 \rightarrow 64$, $62 \rightarrow 61$ on HotpotQA. Effective rank is bounded above by the sample count (90), but the observed 54–58 leaves substantial headroom (a $1.5\times$ growth would still be detectable), so a flat result is genuinely informative rather than an artifact of the cap. End-to-end RL leaves this rank measure unchanged.

Coarse PCA: no shift. Two complementary measurements rule out a coarse population-level shift after RL. *First*, fitting PCA on the before-RL token cloud only and projecting the after-RL tokens onto that basis (the “wrap test”), let $\mu_b, \mu_a \in \mathbb{R}^2$ denote the before-RL and after-RL centroids in this leading-PC plane and σ_b^2, σ_a^2 the corresponding variances. Across all three AIME positions $\|\mu_a - \mu_b\| \leq 0.2$ against a cloud spread of ± 100 s of units, with $\sigma_a^2/\sigma_b^2 \in [0.999, 1.000]$; on HotpotQA, $\|\mu_a - \mu_b\| \in [0.09, 0.20]$ and $\sigma_a^2/\sigma_b^2 \in [0.996, 0.999]$. *Second*, in the supervised plane defined by the probe direction, within each correctness class the before-RL and after-RL points overlap with no detectable translation along w_{before} or w_{after} . End-to-end RL is therefore not a coarse population shift in either the unsupervised or the supervised geometry.

Per-problem deltas: cross-fitted projection is null. To examine whether the per-problem deltas Δ_i are class-coherent in the probe direction, we apply a cross-fitted protocol that avoids in-sample double-use of w . We split problems into 5 stratified folds; on each held-out fold, we project Δ_i onto w trained on the four training folds with target y_{after} . Across positions, we then compute the class-conditional separation $\hat{S} = \hat{E}[\langle \Delta_i, w \rangle \mid y_i = 1] - \hat{E}[\langle \Delta_i, w \rangle \mid y_i = 0]$ and a 20-permutation label-shuffled null. The observed $\hat{S}$ falls within the null $|\hat{S}|$ -p95 envelope at five of six (dataset, position) cells: AIME Analyzer/Solver/Validator give $\hat{S} = 0.057/0.076/0.259$ against null p95 envelopes 0.746/0.727/0.742, and HotpotQA Solver/Validator give $-0.183/0.225$ against 0.429/0.259. The single exception is HotpotQA Analyzer-end ($\hat{S} = 0.751$ vs null p95 = 0.460). The 20-permutation null is a local rather than family-wise threshold and should not be read as a sharp test; with six cells at a 5% nominal level a single positive is close to chance, and we treat the cross-fitted class-conditional projection as null overall while flagging the HotpotQA Analyzer-end cell as a candidate for follow-up with a tighter procedure. The geometric mechanism by which the same-label decodability gain at AIME Solver/Validator arises is therefore *not* captured by simple class-coherent movement of Δ_i along the probe direction.

Bridge to §5.3. Read together, these aggregate diagnostics return null-to-small effects: a same-label-controlled AUROC gain of +6–7 pp at AIME Solver/Validator and +1–3 pp on HotpotQA,

with effective rank, leading-PC geometry, and the cross-fitted single-direction Δ_i projection all flat. The per-flipped-problem analysis of §5.3 on a much larger sample ($n = 267$ to $n = 1038$ depending on cell) is the grain at which RL’s cache-level effect becomes detectable; the mechanism breakdown above shows the cache-level signal aggregates several distinct optimization-aligned behaviors rather than a single “aligned representation” geometry the aggregate probes might have captured.

E Training configuration details

This appendix lists the configuration used for the CacheMAS+RL checkpoints in Table 1. TextMAS, LatentMAS, and CacheMAS are training-free and unaffected by these settings.

Optimization. GRPO [Shao et al., 2024] with PPO clip $\epsilon=0.2$, KL coefficient $\beta=10^{-3}$ to a frozen reference policy, and AdamW at learning rate 10^{-6} with no warmup. Each step processes a global batch of 16 problems $\times n=8$ rollouts = 128 trajectories, with one PPO inner pass over all 128. Rewards are sequence-level scalars, ± 1 correctness via `math_verify` for math, exact-match against gold for multi-hop QA, and the rollout-0 `evalplus` pass for code. No process or per-agent rewards are used.

Rollout sampling. Training rollouts use temperature 1.0 and top- $p=1.0$, with the per-domain decoding budget reported in §4.1 as the response cap. The shared KV cache is held in vLLM PagedAttention via the multi-agent model runner described in Section 3.3.

Data filtering. After each rollout group of $n=8$ trajectories, we drop the prompt if all n are correct or all n are incorrect, keeping only partially-correct groups. This excludes zero-signal groups whose group-normalized advantage would collapse to zero anyway, while preserving the learning signal on the rest.

Compute. Each domain trains for one epoch over the corresponding training set on $4\times A100$ -80GB (math, code) or $4\times A100$ -40GB (multi-hop QA) with TP= 4, using verl [Sheng et al., 2024] on top of vLLM rollout. Wall time is roughly 4–11 hours per domain depending on benchmark length and rollout response distribution.